CS 5402: Introduction to Data Mining

SP2026

(Lecture -11)

Ensemble Method

Dr. Huiyuan Yang

Department of Computer Science
Missouri S&T

https://hyang428.github.io

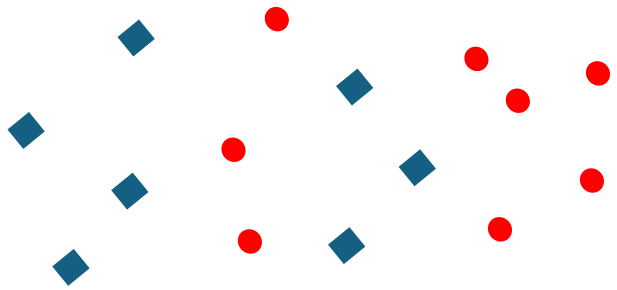
MISSOURI S&T

Why Use an Ensemble?

We've been training *single* models (Linear Regression, Decision Trees, Perceptron).

- A single model might be **overfit** (high variance).
- A single model might be **underfit** (high bias).

Ensemble Learning: The strategy of combining *multiple* models to create a *single, stronger* model.



The goal isn't just to be different—it's to be **better**. Ensembles are how we win machine learning competitions.

Example-1:

Otto Group Product Classification Challenge

Overview | Data | Code | Models | Discussion | Leaderboard | Rules

Description

Get started on this competition through Kaggle Scripts

The Otto Group is one of the world's biggest e-commerce companies, with subsidiaries in more than 20 countries, including Crate & Barrel (USA), Otto.de (Germany) and 3 Suisses (France). We are selling millions of products worldwide every day, with several thousand products being added to our product line.

A consistent analysis of the performance of our products is crucial. However, due to our diverse global infrastructure, many identical products get classified differently. Therefore, the quality of our product analysis depends heavily on the ability to accurately cluster similar products. The better the classification, the more insights we can generate about our product range.

Prizes & Awards

$10,000
Awards Points & Medals

Participation

4,363 Entrants
3,841 Participants
3,507 Teams
43,398 Submissions



https://www.kaggle.com/c/otto-group-product-classification-challenge/overview

For this competition, we have provided a dataset with 93 features for more than 200,000 products. The objective is to build a predictive model which is able to distinguish between our main product categories. The winning models will be open sourced.

Models and features used for 2nd level training:

X = Train and test sets

The winner solution: Ensemble Learning

-Model 1: RandomForest(R). Dataset: X
-Model 2: Logistic Regression(scikit). Dataset: Log(X+1)
-Model 3: Extra Trees Classifier(scikit). Dataset: Log(X+1) (but could be raw)
-Model 4: KNeighborsClassifier(scikit). Dataset: Scale(Log(X+1))
-Model 5: libfm. Dataset: Sparse(X). Each feature value is a unique level.
-Model 6: H2O NN. Bag of 10 runs. Dataset: sqrt(X + 3/8)
-Model 7: Multinomial Naive Bayes(scikit). Dataset: Log(X+1)
-Model 8: Lasagne NN(CPU). Bag of 2 NN runs. First with Dataset Scale(Log(X+1)) and second with Dataset Scale(X)
-Model 9: Lasagne NN(CPU). Bag of 6 runs. Dataset: Scale(Log(X+1))
-Model 10: T-sne. Dimension reduction to 3 dimensions. Also stacked 2 kmeans features using the T-sne 3 dimensions. Dataset: Log(X+1)
-Model 11: Sofia(R). Dataset: one against all with learner_type="logreg-pegasos" and loop_type="balanced-stochastic". Dataset: Scale(X)
-Model 12: Sofia(R). Trainned one against all with learner_type="logreg-pegasos" and loop_type="balanced-stochastic". Dataset: Scale(X, T-sne Dimension, some 3 level interactions between 13 most important features based in randomForest importance)
-Model 13: Sofia(R). Trainned one against all with learner_type="logreg-pegasos" and loop_type="combined-roc". Dataset: Log(1+X, T-sne Dimension, some 3 level interactions between 13 most important features based in randomForest importance)
-Model 14: Xgboost(R). Trainned one against all. Dataset: (X, feature sum(zeros) by row). Replaced zeros with NA.
-Model 15: Xgboost(R). Trainned Multiclass Soft-Prob. Dataset: (X, 7 Kmeans features with different number of clusters, rowSums(X==0), rowSums(Scale(X)>0.5), rowSums(Scale(X)< -0.5))
-Model 16: Xgboost(R). Trainned Multiclass Soft-Prob. Dataset: (X, T-sne features, Some Kmeans clusters of X)
-Model 17: Xgboost(R): Trainned Multiclass Soft-Prob. Dataset: (X, T-sne features, Some Kmeans clusters of log(1+X))
-Model 18: Xgboost(R): Trainned Multiclass Soft-Prob. Dataset: (X, T-sne features, Some Kmeans clusters of Scale(X))
-Model 19: Lasagne NN(GPU). 2-Layer. Bag of 120 NN runs with different number of epochs.
-Model 20: Lasagne NN(GPU). 3-Layer. Bag of 120 NN runs with different number of epochs.
-Model 21: XGboost. Trained on raw features. Extremely bagged (30 times averaged).
-Model 22: KNN on features X + int(X == 0)
-Model 23: KNN on features X + int(X == 0) + log(X + 1)
-Model 24: KNN on raw with 2 neighbours
-Model 25: KNN on raw with 4 neighbours
-Model 26: KNN on raw with 8 neighbours
-Model 27: KNN on raw with 16 neighbours
-Model 28: KNN on raw with 32 neighbours
-Model 29: KNN on raw with 64 neighbours
-Model 30: KNN on raw with 128 neighbours
-Model 31: KNN on raw with 256 neighbours
-Model 32: KNN on raw with 512 neighbours
-Model 33: KNN on raw with 1024 neighbours

Example-2:

Crowdflower Search Results Relevance

Overview Data Code Models Discussion Leaderboard Rules

search algorithms.

Make a first submission with this Python benchmark on Kaggle scripts.



Competition Host
Figure Eight

Prizes & Awards
$20,000
Awards Points & Medals

Participation
1,617 Entrants
1,422 Participants
1,324 Teams
23,177 Submissions

The dataset for this competition was created using query-result pairings enriched on the CrowdFlower platform. They are sponsoring this competition as an investment in the open-source data science community. A dataset collected, cleaned, and labeled by CrowdFlower can make your supervised machine learning dreams come true.

https://www.kaggle.com/c/crowdflower-search-relevance/overview/prizes

Kaggle_CrowdFlower

The winner solution: Ensemble Learning (2)

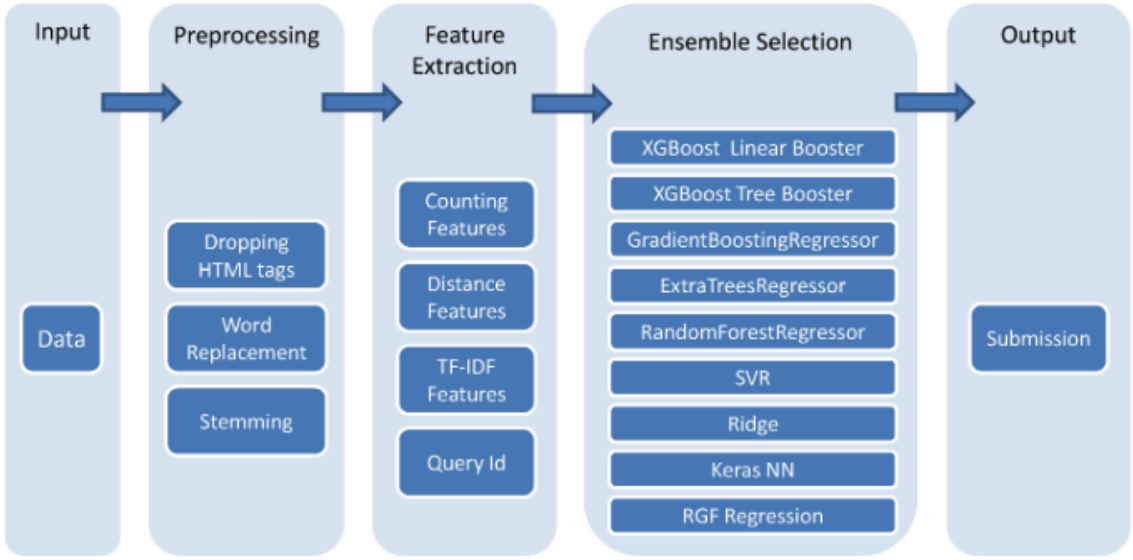
1st Place Solution for Search Results Relevance Competition on Kaggle

The best single model we have obtained during the competition was an XGBoost model with linear booster of Public LB score **0.69322** and Private LB score **0.70768**. Our final winning submission was a median ensemble of 35 best Public LB submissions. This submission scored **0.70807** on Public LB and **0.72189** on Private LB.

What's New

- 2016/05/14: For a more clean and modularized version of this code and framework, you may want to check Kaggle_HomeDepot, which holds the code of Turing Test's solution for the recently Home Depot Product Search Relevance Competition on Kaggle.

FlowChart



https://github.com/ChenglongChen/kaggle-CrowdFlower

Ensemble Learning

Ensemble learning is a machine learning technique that combines multiple models (often called **weak learners** or **base models**) to create a **stronger, more accurate model**. The goal is to reduce variance, bias, or improve generalization.

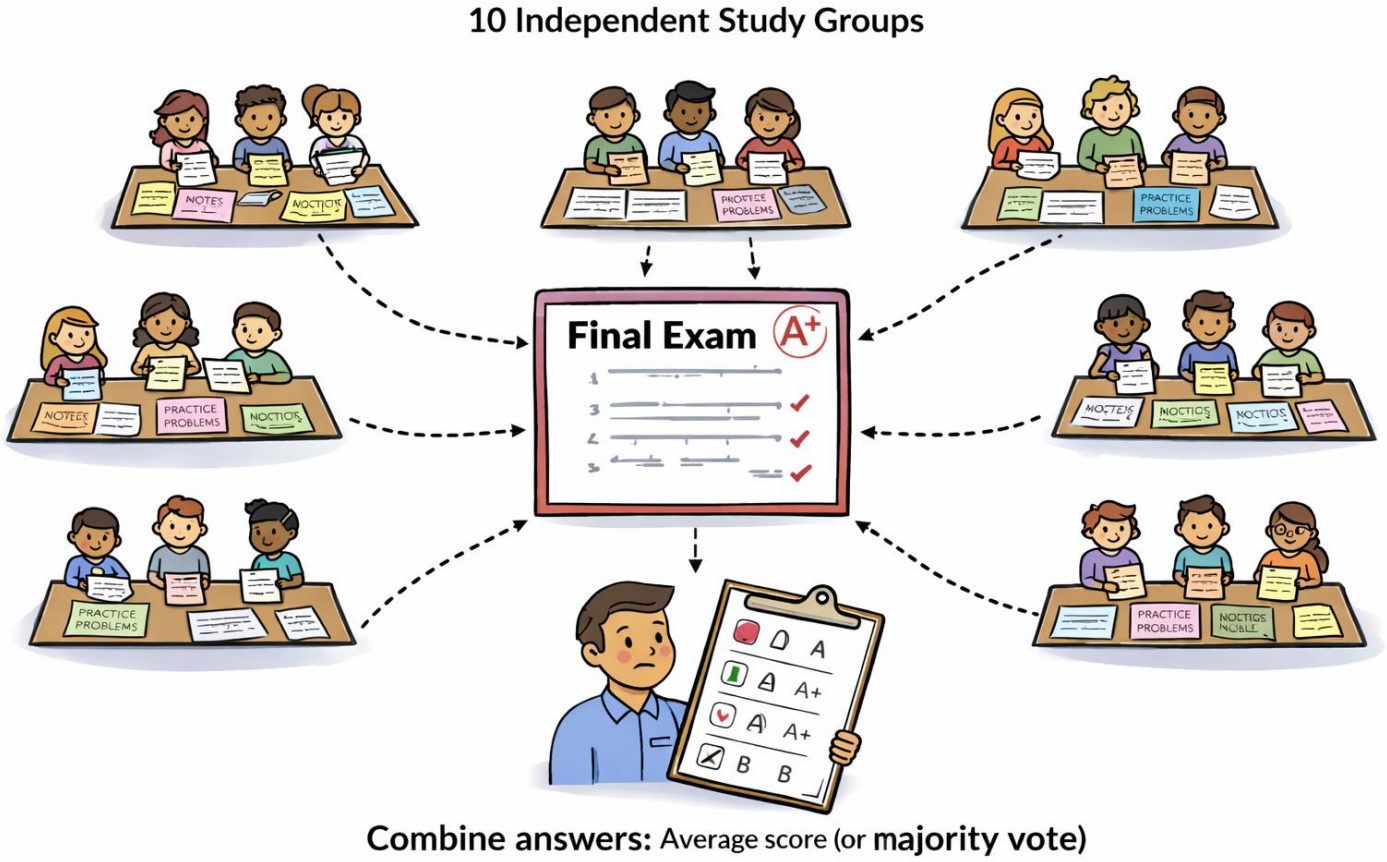
Why Use Ensemble Learning?

- Reduces overfitting (high variance) by averaging multiple predictions.
- Improves accuracy by leveraging different models' strengths.
- Handles complex relationships in data better than individual models.

We will learn the two most powerful ensemble techniques:

- **Bagging (The "Parallel" Approach)**
- **Boosting (The "Sequential" Approach)**

Bagging: The “Parallel Study Group”



The Result: The final "ensembled" answer is far more robust and accurate than any *single* study group's. This method is great at preventing **overfitting**.

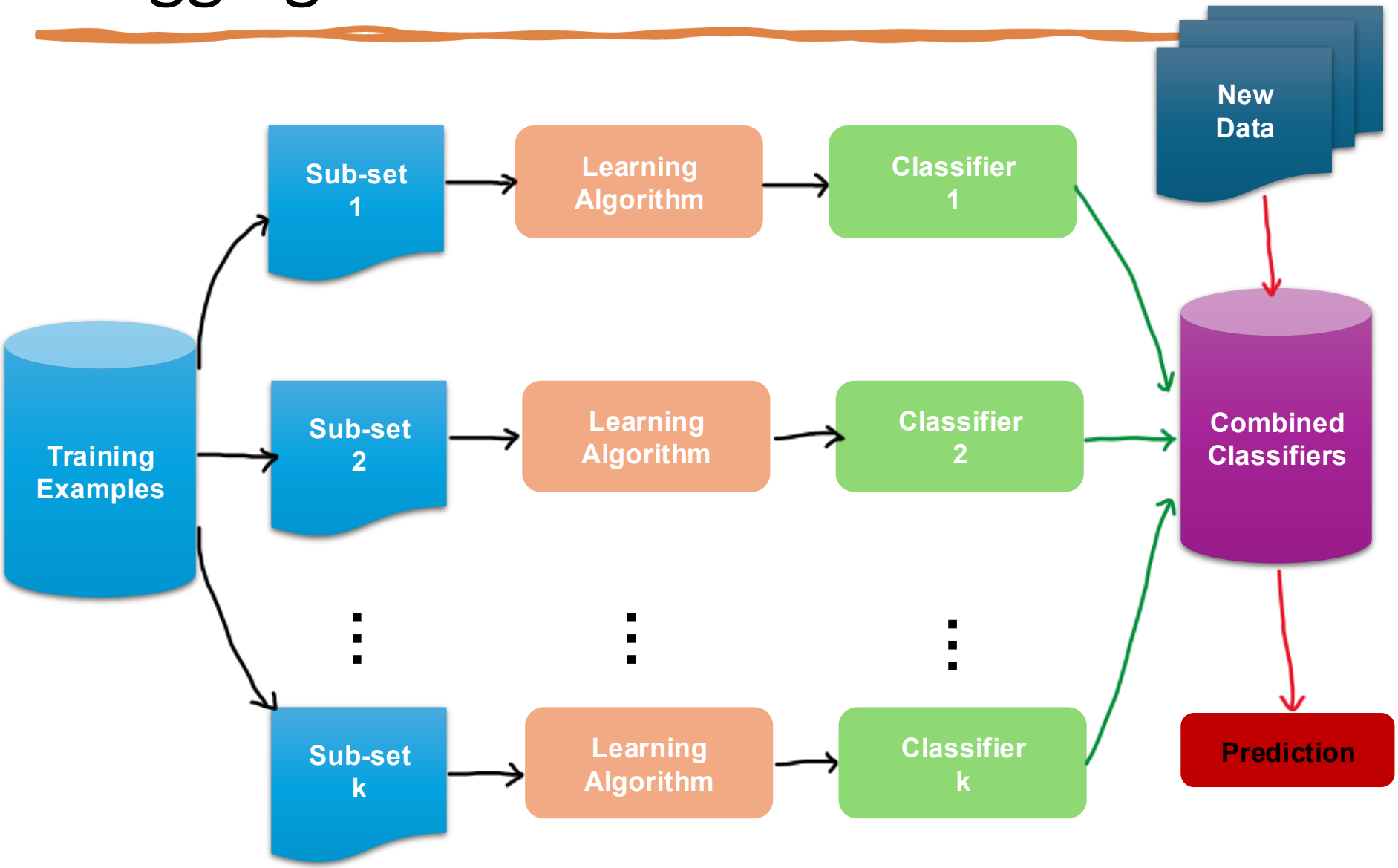
Bagging: How It Technically Works

Bagging = **B**ootstrap **Agg**regating

1. **Bootstrap:** From your original training data (N samples), create k new datasets by **sampling with replacement**.
   - This means some data points will appear multiple times in one dataset, and some won't appear at all!
2. **Train:** Train k *independent* models in **parallel** (e.g., k deep Decision Trees), one on each new dataset.
3. **Aggregate (Vote):** To make a prediction on new data:
   - **Classification:** Each model gets one "vote." The majority class wins.
   - **Regression:** Take the *average* of all model predictions.

Main Goal: To **reduce variance**. It tames unstable, high-variance models (like deep decision trees) and makes them stable.

Bagging: General Idea



The Star of Bagging: The Random Forests

- A **Random Forest** is simply a **Bagging** algorithm using **Decision Trees**.
- One of the typical algorithms in Bagging.

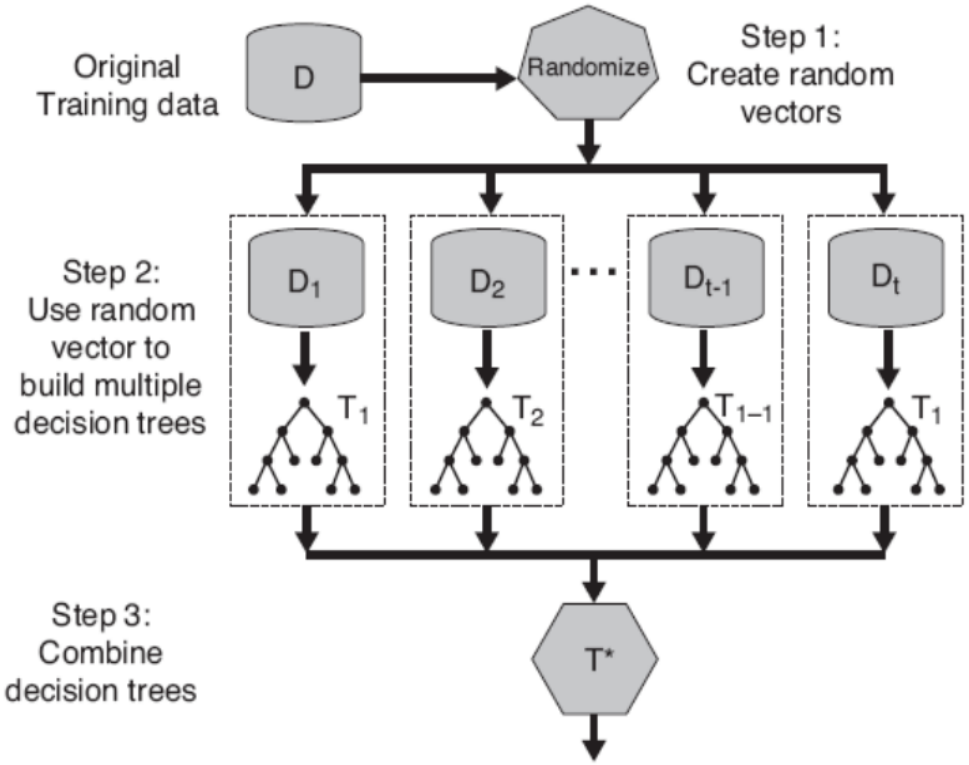
It has *one extra trick* to make it even more powerful:

- When building each tree, at *every split*, the tree is only allowed to consider a *random subset of features*.

Why?

- This prevents all the trees from just latching onto the one strongest feature (e.g., "Square Footage" for house prices).
- It forces the trees to be diverse and learn different patterns.
- A more diverse "crowd" gives a smarter final answer.

Random Forests



Introduce two sources of randomness: "Bagging" and "Random input Vectors"

- Bagging method: each tree is grown using a bootstrap sample of training data;
- Random vector method: at each node, best split is chosen from a random sample of m attributes instead of all attributes

Random Forest

1. For $b = 1$ to B:
   (a) Draw a bootstrap sample $\mathbf{Z}^*$ of size N from the training data.
   (b) Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.
      i. Select m variables at random from the p variables.
      ii. Pick the best variable/split-point among the m.
      iii. Split the node into two daughter nodes.
2. Output the ensemble of trees $\{T_b\}_1^B$.

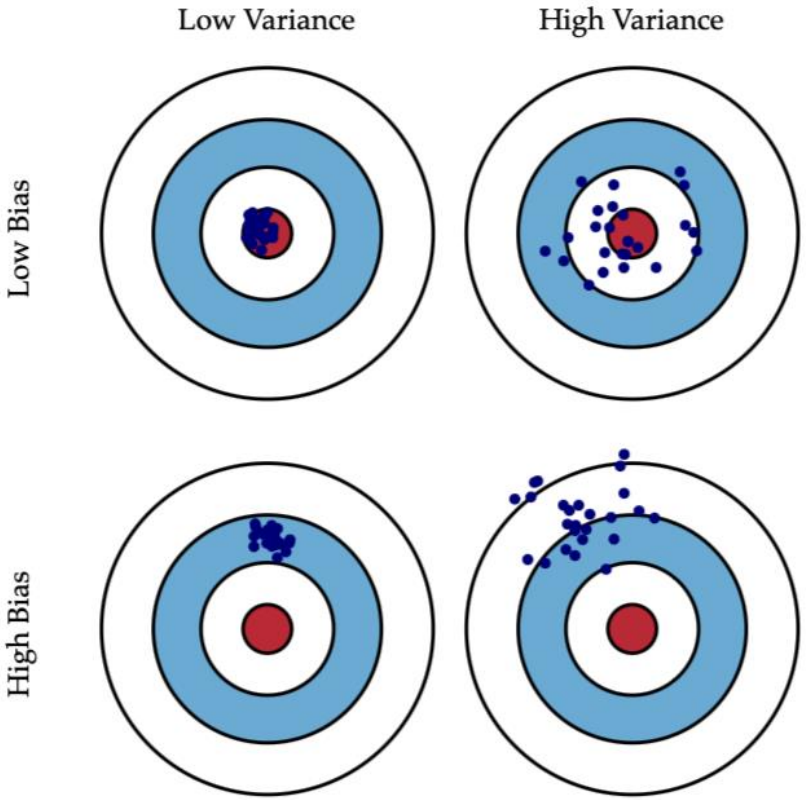
To make a prediction at a new point x:

- Regression: $\hat{f}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$
- Classification: Let $\hat{C}_b(x)$ be the class prediction of the b-th random forest tree. Then $\hat{C}(x) = majority\ vote\ \{\hat{C}_b(x)\}_{b=1}^{B}$

Review: Bias-Variance Tradeoff

GT: $f(x)$
Estimated: $\hat{f}(x)$

Bias = $E\left[\hat{f}(x)\right] - f(x)$
Variance = $E\left[\left(\hat{f}(x) - E\left[\hat{f}(x)\right]\right)^2\right]$



Further reading: http://scott.fortmann-roe.com/docs/BiasVariance.html

Bias-Variance Tradeoff

Reduce variance without increasing bias:

- Averaging reduces variance:

$$Var(\bar{X}) = \frac{Var(X)}{N}$$

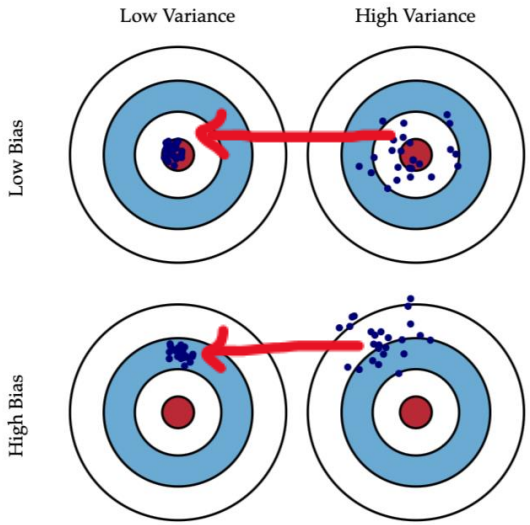
(When predictions are independent)

Average models to reduce model variance.

Bagging: bias-variance perspective

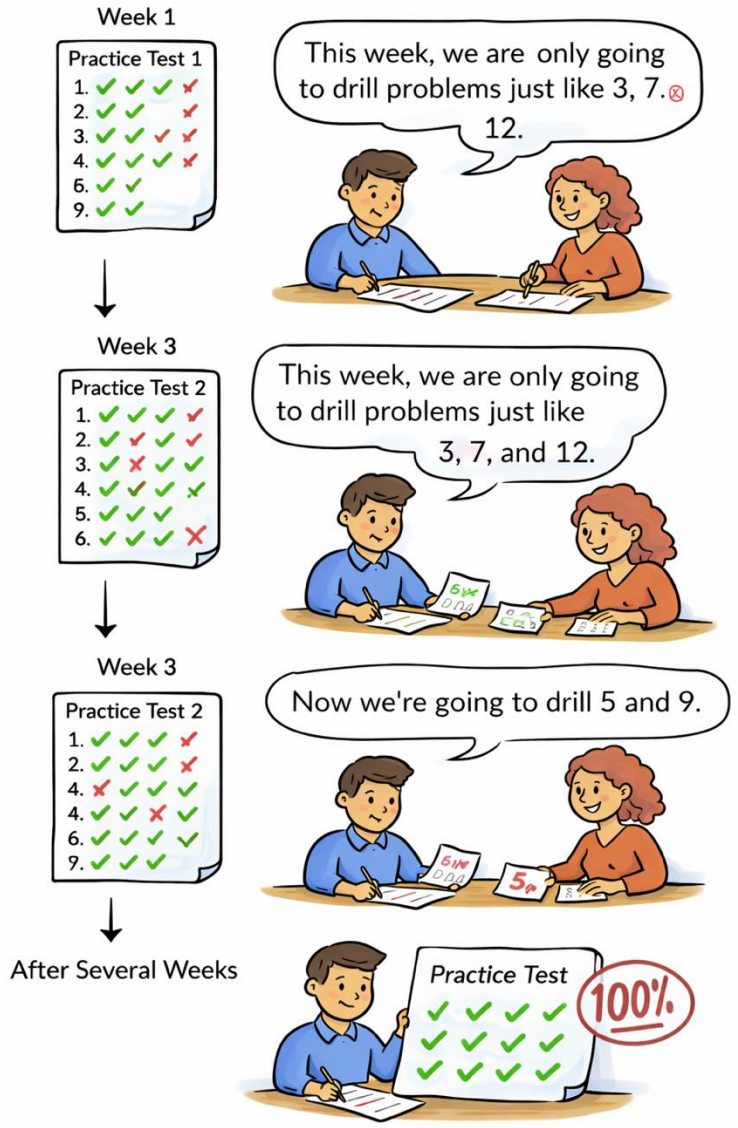
Does bagging reduce Bias while decreasing variance?

- Bagging reduces variance by averaging
- Bagging has little effect on bias.



Boosting: The “Intensive Tutor”

The Result: Your final skill is a *weighted combination* of all these study sessions, focused on correcting your mistakes.



Boosting

- Idea: given a weak learner, run it multiple times on (reweighted) training data, then let the learned classifiers vote
- On each iteration t:
  - Weight each training example by how incorrectly it was classified
  - Learn
    - a hypothesis – h_t
    - A strength for this hypothesis – α_t
- Final classifier:
  - A linear combination of the votes of the different classifiers weighted by their strength

$$f(x) = \sum_t \alpha_t \cdot h_t(x)$$

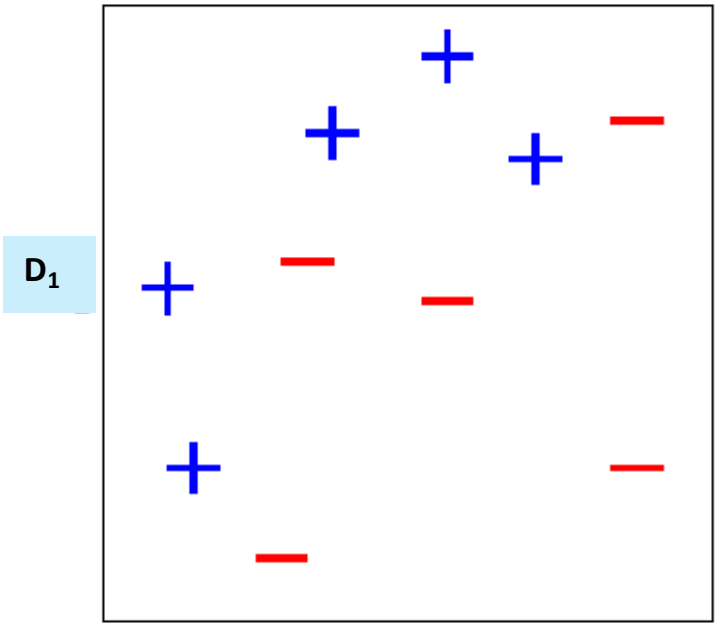
Boosting: How It Technically Works

Boosting works **sequentially** to turn many "weak learners" (models just *barely* better than random) into one *very* strong model.

1. **Start:** Train a simple "weak" model (like a 1-level decision tree, or "stump") on the data.
2. **Find Mistakes:** Identify which data points the model got **wrong**.
3. **Increase Weights:** Give *more importance* ("weight") to the misclassified data points.
4. **Train New Model:** Train a *new* weak model that tries *extra hard* to classify these newly weighted, "difficult" points correctly.
5. **Repeat:** Repeat steps 2-4 many times (e.g., 100+).
6. **Final Prediction:** All models vote, but the models that performed better get a *bigger say* (a weighted vote).

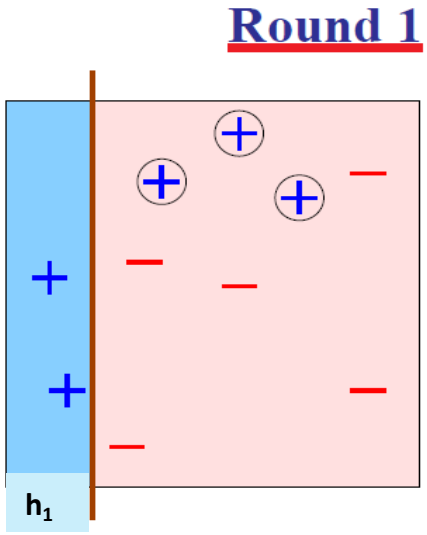
Main Goal: To **reduce bias**. It takes simple, underfit models and *boosts* them into a complex, accurate one.

A Toy Example (1)



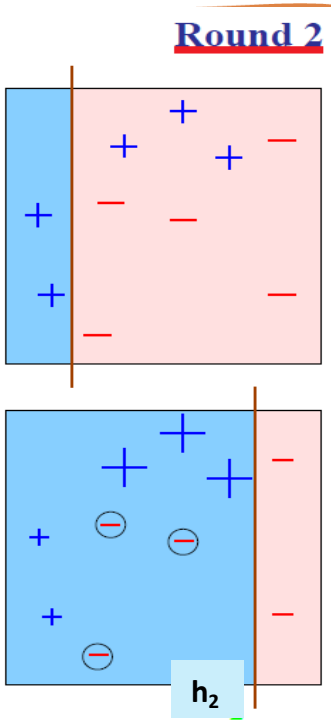
Hypothesis: $f(x) = \begin{cases} +, & x < v \\ -, & x \geq v \end{cases}$ or $f(x) = \begin{cases} -, & x < v \\ +, & x \geq v \end{cases}$

A Toy Example (2)



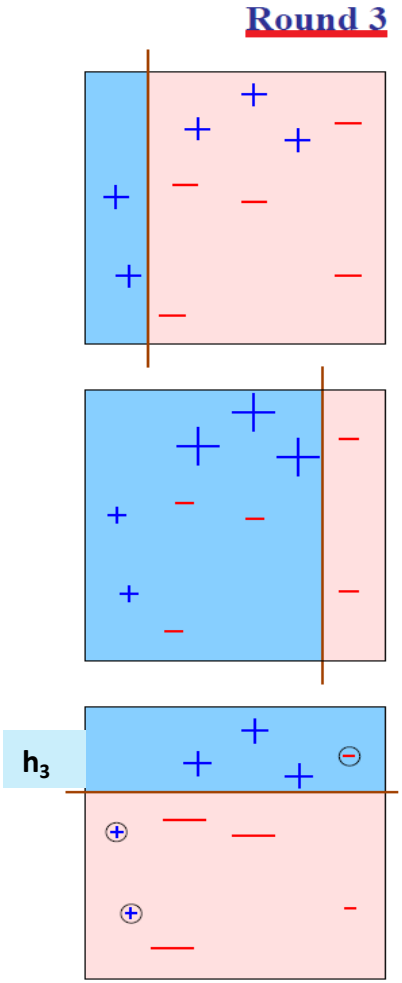
$\varepsilon_1 = 0.3$
$\alpha_1 = 0.42$

A Toy Example (3)



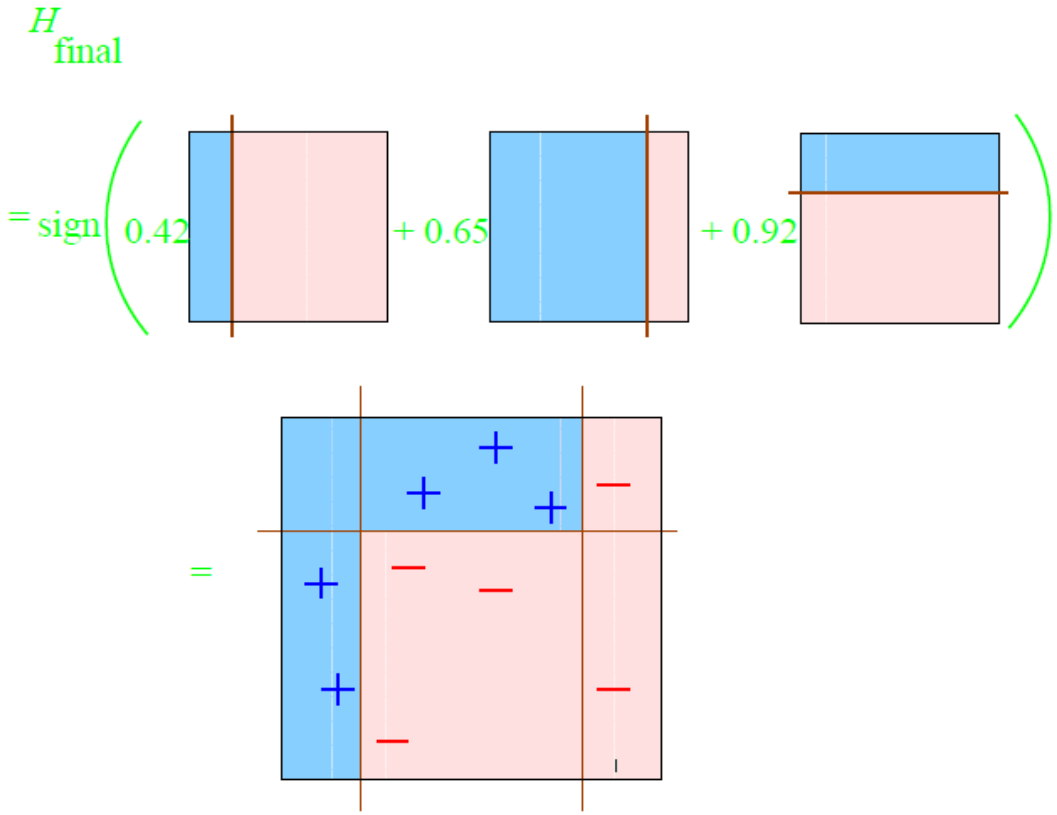
$\varepsilon_2 = 0.21$
$\alpha_2 = 0.65$

A Toy Example (4)



$\varepsilon_3 = 0.14$
$\alpha_3 = 0.92$

A Toy Example (5)



Boosting:

Two key issues:

I. How to adjust the weight or probability distribution of the training data in each round?

By increasing the weights of the misclassified examples by weak classifiers, and decreasing the weights of correctly classified examples in the previous round, the classifier will achieve better performance on misclassified data;

II. How are weak classifiers combined?

Weak classifiers are combined through additive models (i.e., AdaBoost combines them through weighted combination), increasing the weight of classifiers with lower error rates while decreasing the weight of those higher error rates.

Famous Boosting Algorithms

- AdaBoost (Adaptive Boosting):
  - The classic example.
  - It "adapts" by re-weighting the data points that were misclassified.
- Gradient Boosting (GBM):
  - The modern workhorse.
  - Instead of re-weighting data, the next model is trained to predict the residual error of the previous model.
  - Analogy: Model 1 is off by $50k. Model 2's job is to predict that "$50k" error. Model 3 predicts Model 2's error, and so on.
- XGBoost (Extreme Gradient Boosting):
  - A highly optimized, faster, and more regularized version of Gradient Boosting.
  - This (and its cousins, LightGBM) is what wins most modern data science competitions on tabular data.

AdaBoost (1)

Input: training data $T = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$, where $x_i \in X, y_i \in Y = \{-1, +1\}$, and weak learners
Output: final classifier $H(x)$

- (Sometimes) not all data points are equal
  - Some data points are more important than others
- Consider a weighted dataset:
  - $D(i)$: weight of the ith training example (x_i, y_i)
  - Interpretations: i^{th} training example counts as $D(i)$ examples;

$D_t(i)$: weight of the ith training example (x_i, y_i) at iteration-t.

AdaBoost (2)

Initialize: $D_1(i) = \frac{1}{n}$

For $t = 1, \ldots, T$:

- Train weak classifier $h_t(x)$ using distribution D_t

$$h_t(x): X \to \{-1, +1\}$$

- Calculate the training error of current weak classifier $h_t(x)$:

$$\epsilon_t = \sum_{i=1}^{n} D_t(i) I(h_t(x_i) \neq y_i)$$

- Choose coefficient α_t:

$$\alpha_t = \frac{1}{2} ln\left(\frac{1 - \epsilon_t}{\epsilon_t}\right)$$

- Update weights D_{t+1}:

$$D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

Where Z_t is a normalization factor:

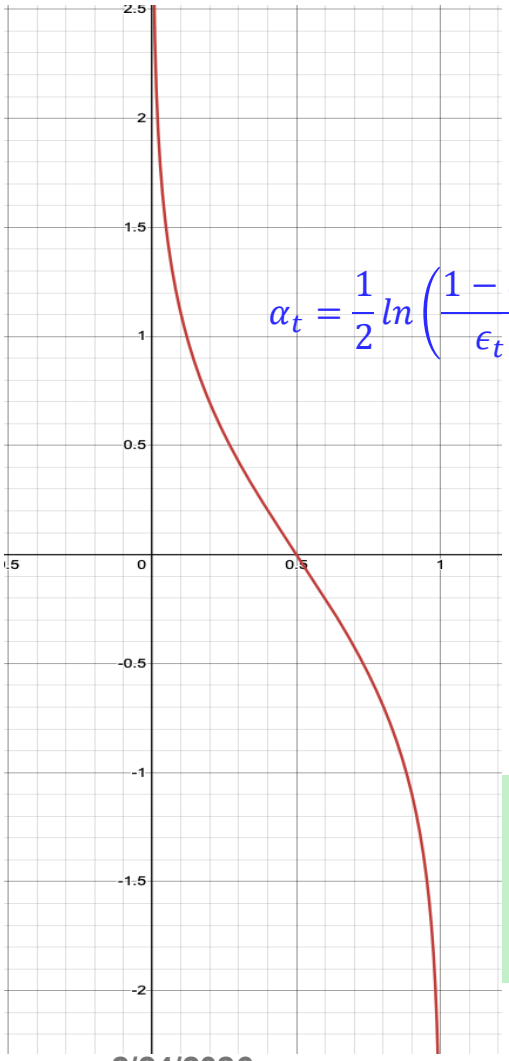
$$Z_t = \sum_{i=1}^{n} D_t(i)\exp(-\alpha_t y_i h_t(x_i))$$

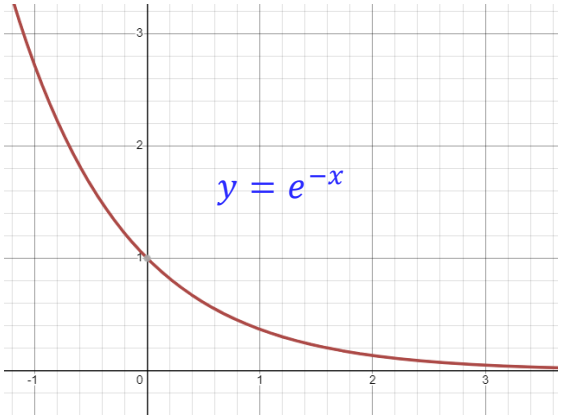
Output the final classifier:

$$H(x) = sign\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$$

AdaBoost (3)

$$\epsilon_t = \sum_{i=1}^{n} D_t(i) I(h_t(x_i) \neq y_i)$$

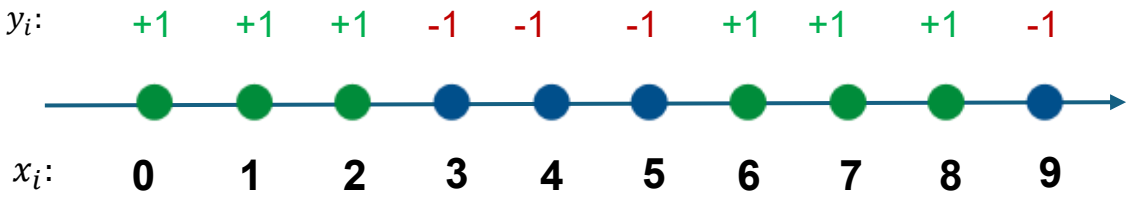




$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

$$H(x) = sign\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$$

Example: Adaboost (1)



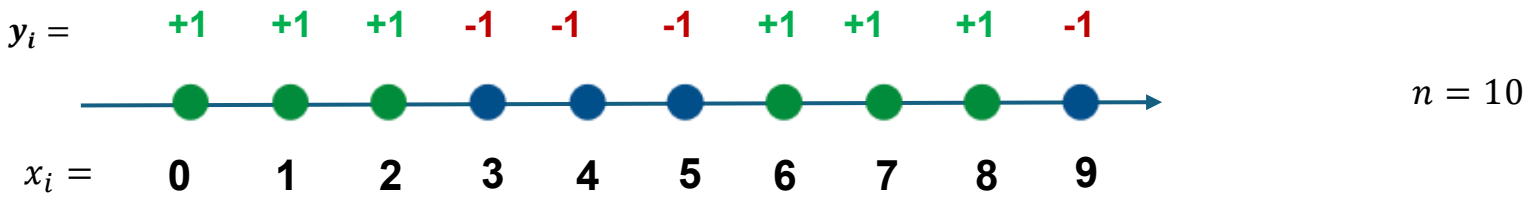
- Assume that hypothesis for weak classifier is:

$$x > v \; or \; x < v,$$

- and threshold v makes the classifier achieves the lowest classification error on the training dataset.

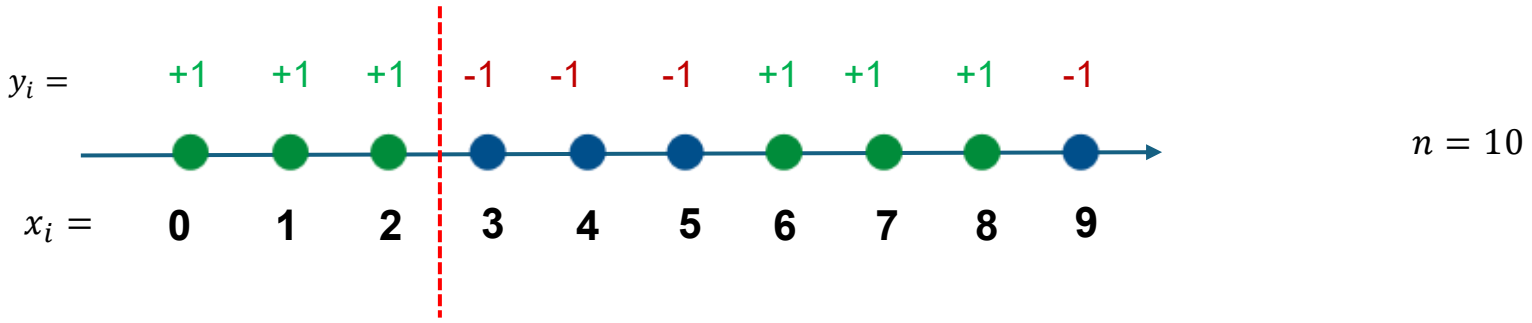
- Find a strong classifier using AdaBoost.

Example: Adaboost (2)



Initialize weight: $D_1 = (0.1, 0.1, \ldots, 0.1)$

Example: Adaboost (3)



Initialize weight: $D_1 = (0.1, 0.1, \ldots, 0.1)$

For iteration-1:

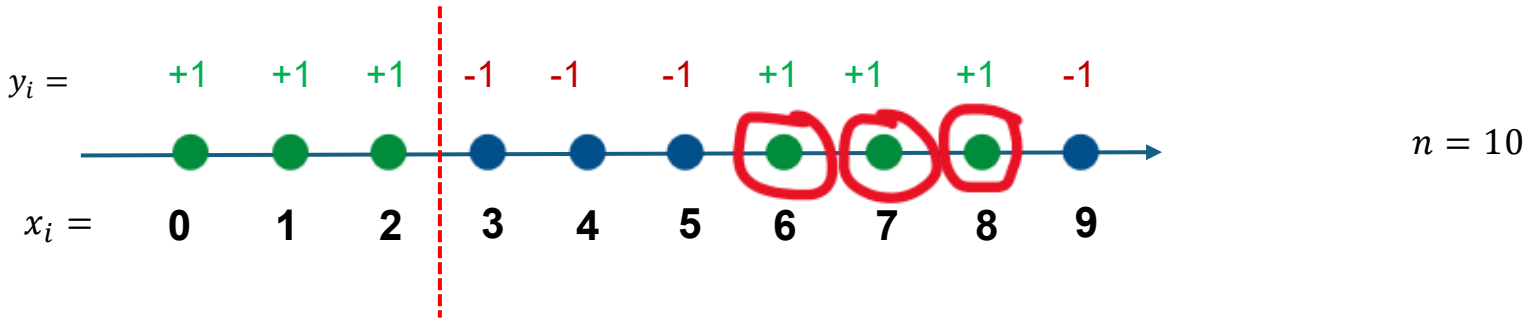
I. Train weak classifier $h_1(x)$ using distribution D_1:

The classification error rate is minimized when the threshold v is set to 2.5.

Therefore, the base classifier is:

$$h_1(x) = \begin{cases} 1, & x < 2.5 \\ -1, & x \geq 2.5 \end{cases}$$

Example: Adaboost (4)



Initialize weight: $D_1 = (0.1, 0.1, \ldots, 0.1)$

For iteration-1:

I. Train weak classifier $h_1(x)$ using distribution D_1:

The classification error rate is minimized when the threshold v is set to 2.5.

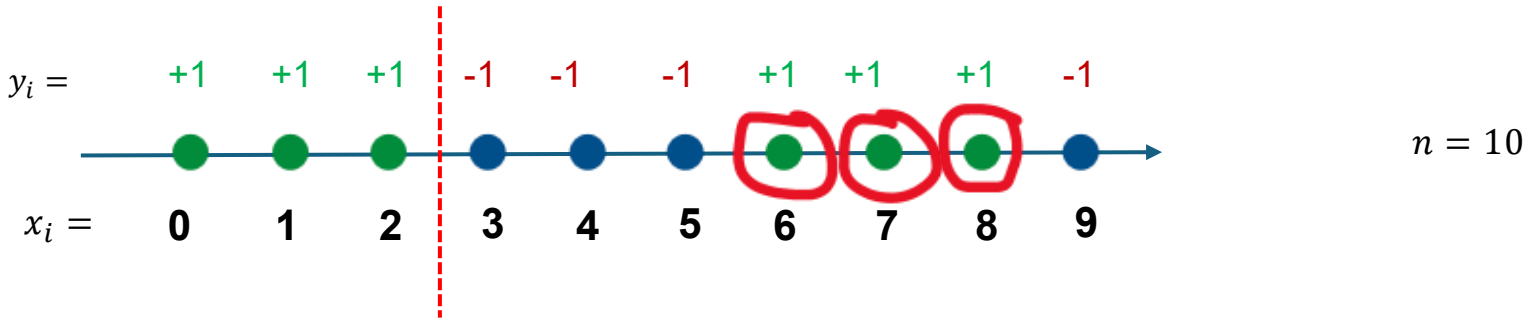
Therefore, the base classifier is:

$$h_1(x) = \begin{cases} 1, & x < 2.5 \\ -1, & x \geq 2.5 \end{cases}$$

II. Calculate the training error of current weak classifier $h_1(x)$:

$$\epsilon_1 = \sum_{i=1}^{10} D_t(i) I(h_1(x_i) \neq y_i) = 0.1 + 0.1 + 0.1 = 0.3$$

Example: Adaboost (5)



Initialize weight: $D_1 = (0.1, 0.1, \ldots, 0.1)$

For iteration-1:

I. Train weak classifier $h_1(x)$ using distribution D_1:

The classification error rate is minimized when the threshold v is set to 2.5.

Therefore, the base classifier is:

$$h_1(x) = \begin{cases} 1, & x < 2.5 \\ -1, & x \geq 2.5 \end{cases}$$

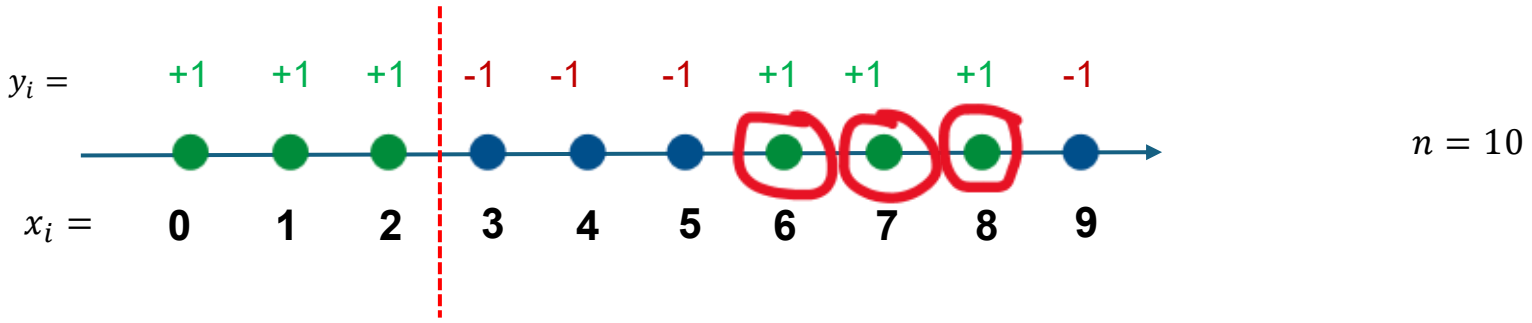
II. Calculate the training error of current weak classifier $h_1(x)$:

$$\epsilon_1 = \sum_{i=1}^{10} D_1(i) I(h_1(x_i) \neq y_i) = 0.1 + 0.1 + 0.1 = 0.3$$

III. Calculate coefficient α_1:

$$\alpha_1 = \frac{1}{2} ln\left(\frac{1-\epsilon_1}{\epsilon_1}\right) = 0.4236$$

Example: Adaboost (6)



Initialize weight: $D_1 = (0.1, 0.1, \ldots, 0.1)$

For iteration-1:

IV. Update weights D_2:

$$D_2(i) = \frac{D_1(i)\exp(-\alpha_1 y_i h_1(x_i))}{Z_1}$$

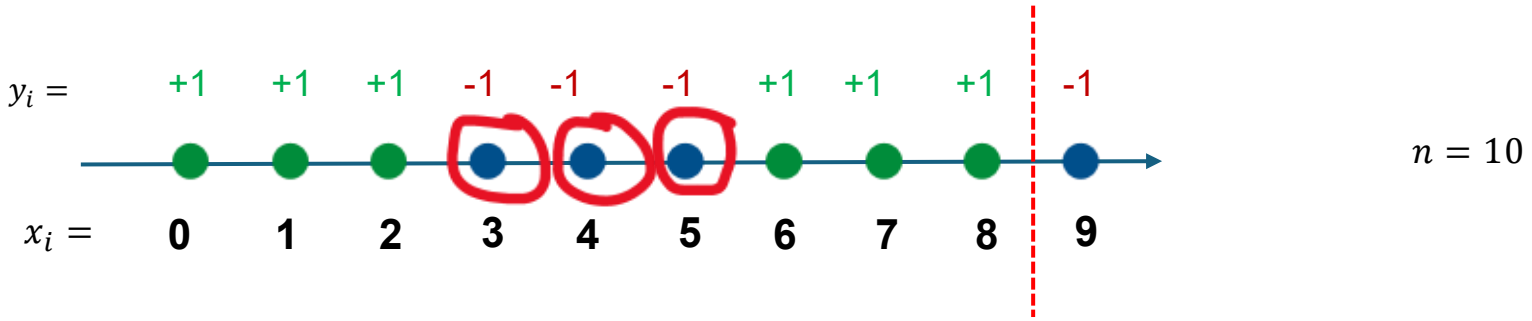
$$D_2 = (0.07143, 0.07143, 0.07143, 0.07143, 0.07143, 0.07143, 0.16667, 0.16667, 0.16667, 0.07143)$$

The number of misclassified points by:

$$f(x) = 0.4236 * h_1(x)$$

is 3.

Example: Adaboost (7)



Initialize weight: $D_1 = (0.1, 0.1, \ldots, 0.1)$

For iteration-2:

I. Train weak classifier $h_2(x)$ using distribution D_2:

The classification error rate is minimized when the threshold v is set to 8.5.

Therefore, the base classifier is:

$$h_2(x) = \begin{cases} 1, & x < 8.5 \\ -1, & x \geq 8.5 \end{cases}$$

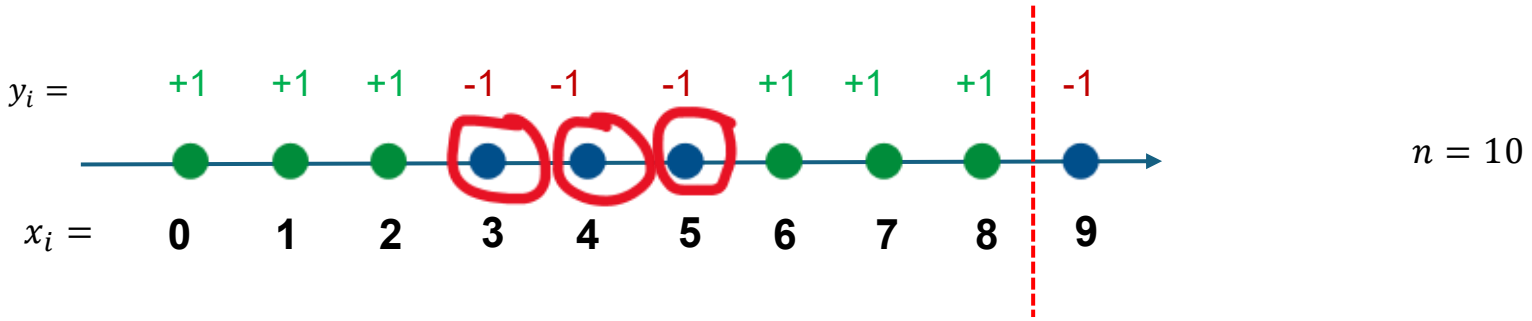
II. Calculate the training error of current weak classifier $h_2(x)$:

$$\epsilon_2 = \sum_{i=1}^{10} D_2(i) I(h_2(x_i) \neq y_i) = 0.07143 + 0.07143 + 0.07143 = 0.2143$$

III. Calculate coefficient α_1:

$$\alpha_2 = \frac{1}{2} ln\left(\frac{1-\epsilon_2}{\epsilon_2}\right) = 0.6496$$

Example: Adaboost (8)



Initialize weight: $D_1 = (0.1, 0.1, \ldots, 0.1)$

For iteration-2:

IV. Update weights D_3:

$$D_3(i) = \frac{D_2(i)\exp(-\alpha_2 y_i h_2(x_i))}{Z_2}$$

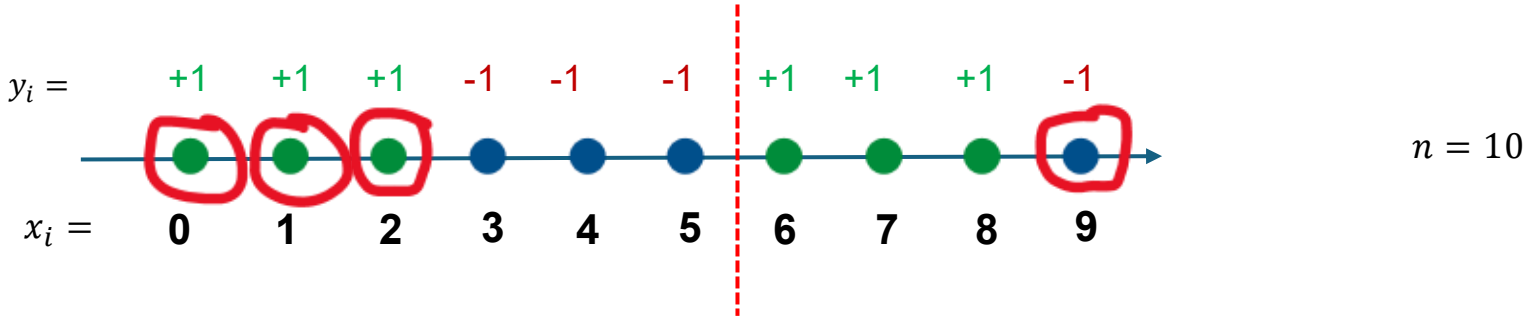
$$D_3 = (0.0455, 0.0455, 0.0455, 0.16667, 0.16667, 0.16667, 0.1060, 0.1060, 0.1060, 0.0455)$$

The number of misclassified points by:

$f(x) = 0.4236 * h_1(x) + 0.6496 * h_2(x)$

is 3.

Example: Adaboost (9)



Initialize weight: $D_1 = (0.1, 0.1, \ldots, 0.1)$

For iteration-3:

I. Train weak classifier $h_3(x)$ using distribution D_3:

The classification error rate is minimized when the threshold v is set to 5.5.

Therefore, the base classifier is:

$$h_3(x) = \begin{cases} -1, & x < 5.5 \\ 1, & x \geq 5.5 \end{cases}$$

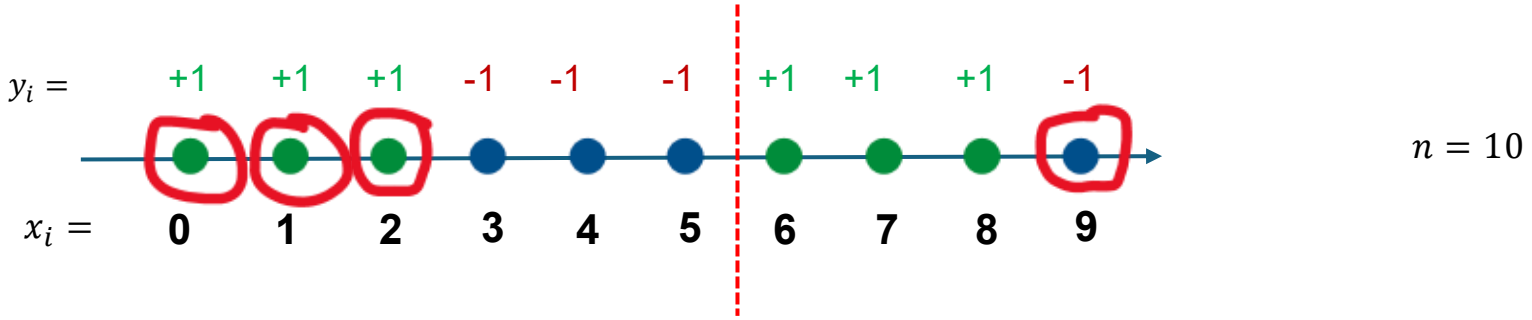
II. Calculate the training error of current weak classifier $h_3(x)$:

$$\epsilon_3 = \sum_{i=1}^{10} D_3(i) I(h_3(x_i) \neq y_i) = 0.0455 + 0.0455 + 0.0455 + 0.0455 = 0.1820$$

III. Calculate coefficient α_3:

$$\alpha_3 = \frac{1}{2} ln\left(\frac{1-\epsilon_3}{\epsilon_3}\right) = 0.7514$$

Example: Adaboost (10)



Initialize weight: $D_1 = (0.1, 0.1, \ldots, 0.1)$

For iteration-3:

IV. Update weights D_4:

$$D_4(i) = \frac{D_3(i)\exp(-\alpha_3 y_i h_3(x_i))}{Z_3}$$

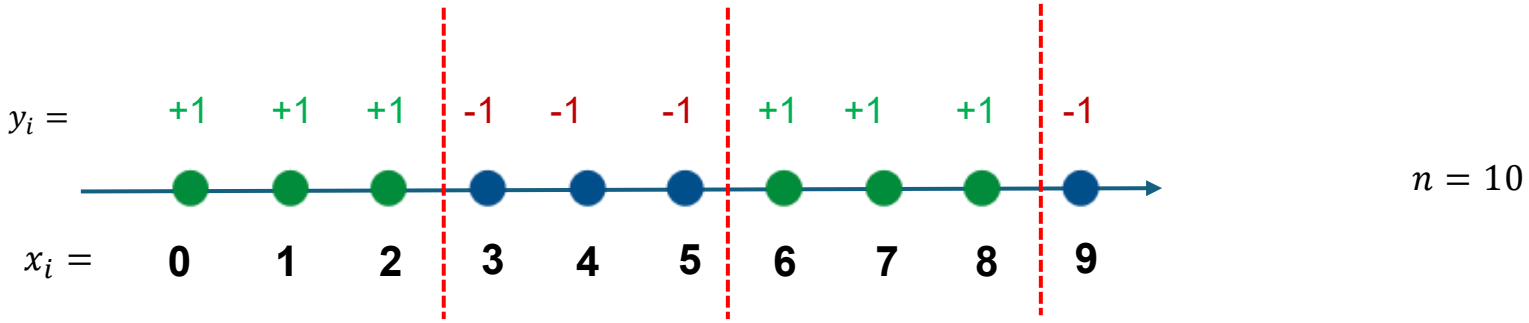
$$D_4 = (0.125, 0.125, 0.125, 0.102, 0.102, 0.102, 0.065, 0.065, 0.065, 0.125)$$

The number of misclassified points by:

$$f(x) = 0.4236 * h_1(x) + 0.6496 * h_2(x) + 0.7514 * h_3(x)$$

is 0.

Example: Adaboost (11)



Final classifier:

$$f(x) = 0.4236 * h_1(x) + 0.6496 * h_2(x) + 0.7514 * h_3(x)$$

Q1:

Which of the following statements about **AdaBoost** is **TRUE**?

A. AdaBoost trains all weak learners independently and combines their predictions equally.

B. AdaBoost assigns higher weights to misclassified samples, forcing the next weak learner to focus on them.

C. AdaBoost is a bagging-based ensemble method that reduces variance.

D. AdaBoost works only with decision trees and cannot use other weak learners.

Q2

Which of the following statements correctly distinguishes AdaBoost from Random Forest?

A. AdaBoost is a boosting algorithm that trains weak learners sequentially, while Random Forest is a bagging algorithm that trains decision trees independently in parallel.

B. Both AdaBoost and Random Forest use the same weighting mechanism for combining models.

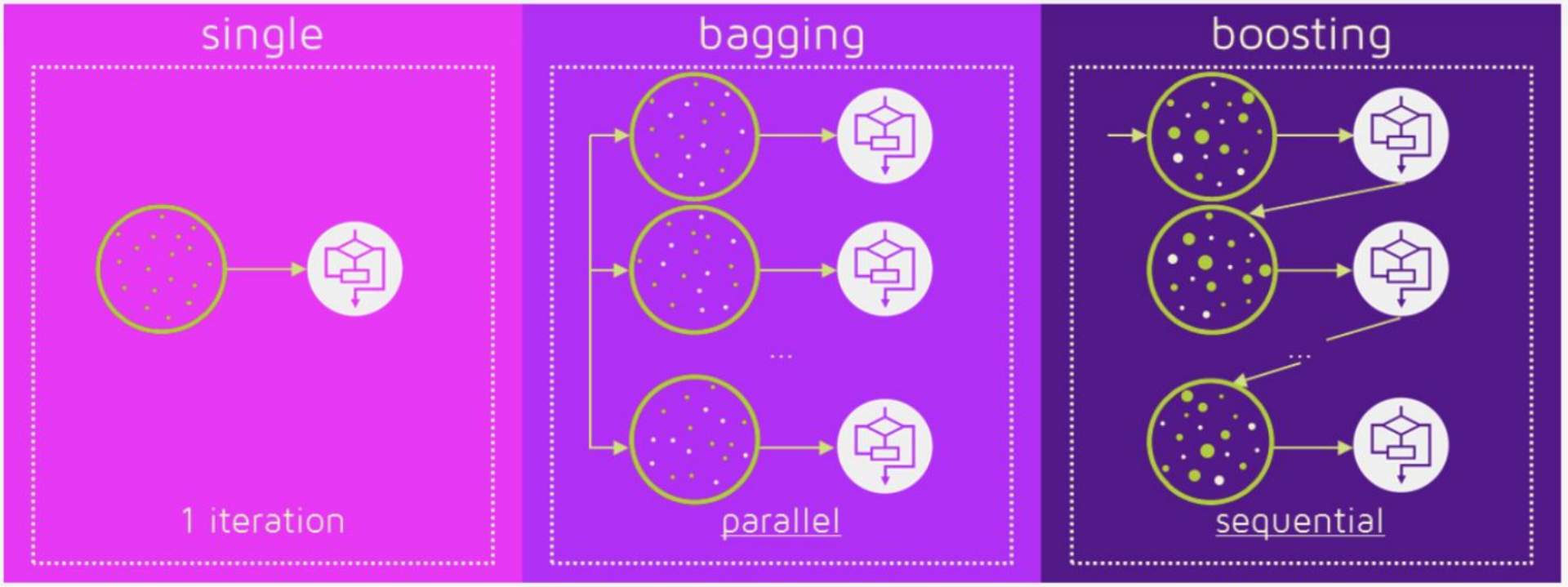
C. Random Forest tends to overfit more than AdaBoost because it relies on a single strong learner.

D. AdaBoost and Random Forest both train models in parallel without any dependencies between them.

Bagging vs. Boosting

	Bagging	Boosting
Sample selection	Different training data	Same training data
Sample weight	Equal weight	Different weights (adjust the weights of examples continuously based on the error rate)
Final output	Equal weight	Different weights (each weak classifier is assigned a corresponding weight)
Parallel computing	YES	NO (Sequential models)

Bagging vs. Boosting (2)



i.e., Random Forest

i.e., Adaboost

After-Class Questions: Conceptual Understanding

1. Explain the main idea of an **ensemble method** in machine learning.
2. Compare **Bagging** and **Boosting** in terms of:
   - how training data are sampled,
   - how base learners are combined,
   - their primary goal (variance reduction vs. bias reduction).

After-Class Questions: Bias–Variance Perspective

Consider a decision tree with unlimited depth as the base learner.

1. Describe the **bias** and **variance** characteristics of a single deep decision tree.
2. Explain how **Bagging** (e.g., Random Forest) changes the bias–variance tradeoff.
3. Explain how **Boosting** (e.g., AdaBoost) changes the bias–variance tradeoff.
4. In which situations would boosting be **more likely to overfit** than bagging? Why?

After-Class Questions: Bagging vs. Boosting

For each of the following scenarios, state whether **Bagging** or **Boosting** is more appropriate and justify your answer:

1. The dataset is small and noisy.
2. The base model is highly unstable.
3. The model consistently underfits the data.
4. Interpretability is a key concern.